

Visão geral - Cenário

O cenário envolve 2 agentes, A e B, cada um deles muda o estado dependendo de quantos ticks se passaram. O tick representa uma unidade de tempo da simulação. Cada agente funciona de forma independente, tendo ciclos próprios de comportamento. O cenário acaba após 25 ticks.

Os dois agentes executam seus estados simultaneamente, mas de maneira independente. Por isso, em um mesmo tick, cada agente pode estar realizando uma ação diferente. Por exemplo, enquanto o Agente A está buscando um alvo, o Agente B pode estar em estado de Alerta.

Visão Geral - Agente

Agente A

O *Agente A* transiciona entre três estados: “EstadoAtacando”, “EstadoBuscando”, “EstadoRepouso”, cada um deles tem um limite de quantos ticks ele pode durar, sendo:

- EstadoAtacando: 2
- EstadoBuscando: 4
- EstadoRepouso: 3

Após o estado passar do limite de ticks, ele vai transicionar para o próximo estado, após finalizar o último (Atacando), ele irá voltar para o Repouso.

Agente B

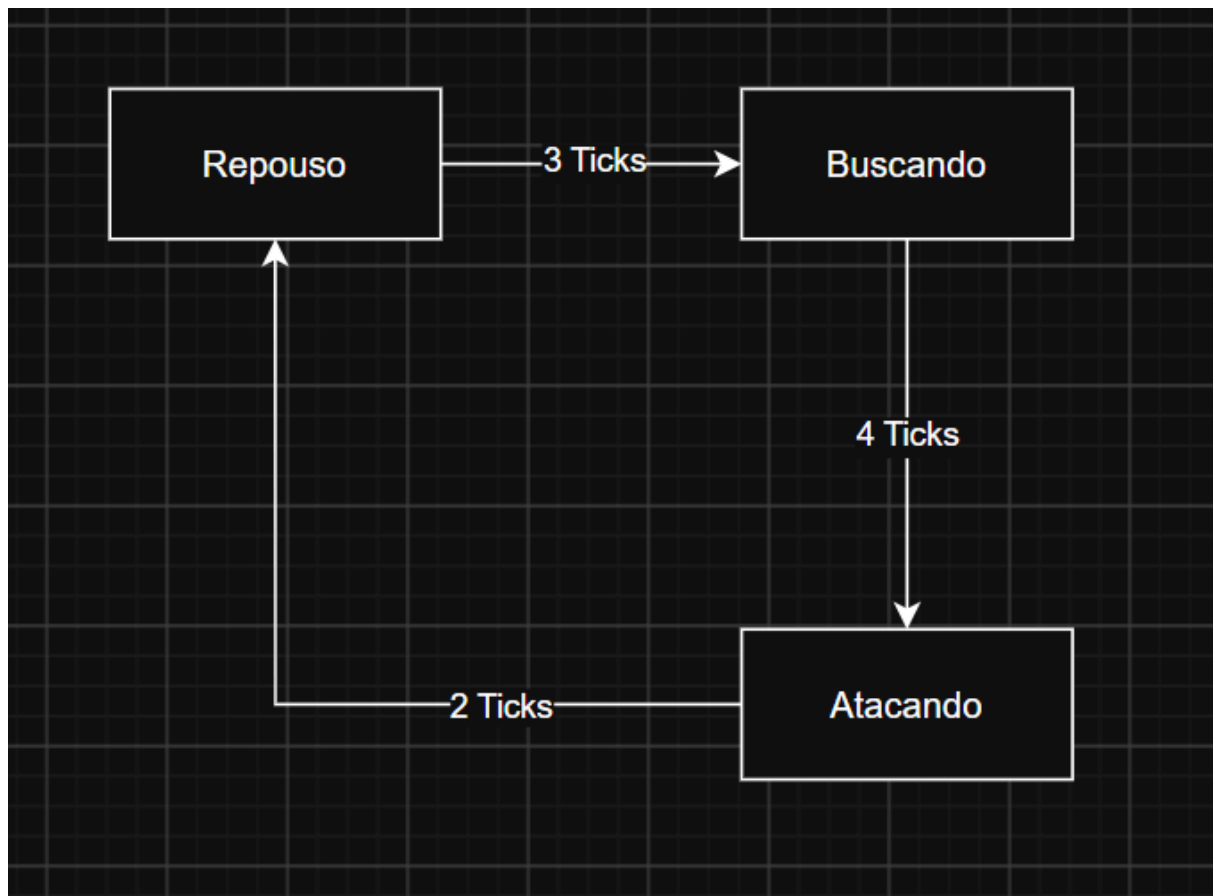
O *Agente B* funciona de forma semelhante ao A, porém eles possuem estados e parâmetros diferentes. Ele transiciona entre dois estados: “EstadoAlerta” e “EstadoPatrulhando”.

- EstadoAlerta: 2
- EstadoPatrulhando: 5

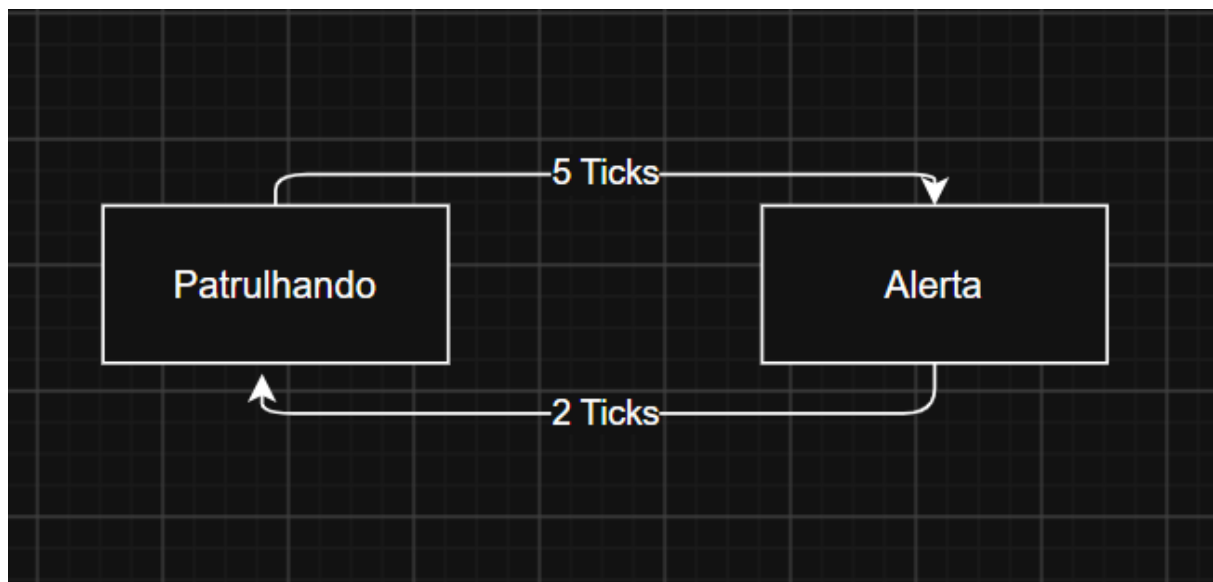
Após o estado passar do limite de ticks, ele vai transicionar para o próximo estado.

Diagrama De Estados

Agente A



Agente B



Regras

Estados A: EstadoAtacando, EstadoBuscando, EstadoRepouso.

Estados B: EstadoAlerta, EstadoPatrulhando.

Ações de saída: Indica que um agente completou um estado.

Ações de entrada: Indica que o agente entrou em um estado.

Execução: A execução da *Máquina de Estados* acontece na classe *Gerenciador*, que cria os agentes ao iniciar o código. Depois disso, a classe imprime o atual estado dos agentes e usa *tick()* para aumentar a contação de ticks.

Transições: As transições são feitas através da contagem de ticks. A variável tick dos agentes é aumentada pela função *tick()*, e quando eles têm a quantidade máxima de ticks do seu estado, a função *transicionarPara()* altera o estado.

Variáveis e Limiares

Agente

Ambos os agentes têm a mesma classe, que são inicializadas em estados diferentes:

- nome(string): Começa nula, e depois guarda o nome do agente (A ou B)
- ticksTotais(int): Começa como zero, e guarda a quantidade de ticks que foram executados.
- estado(Estado): Guarda o estado atual do agente.

Estados

Os estados possuem variáveis que são usadas para a troca de estados:

- LIMITE_TICKS(int): funciona como limiar dos estados, o valor dele é diferente para cada um. (Valores na visão geral)
- contador(int): Guarda a quantidade de ticks que foram executados no estado.
- agente(Agente): Guarda o agente que está no estado.

Gerenciador

As variáveis encontradas no gerenciador, que executa as tarefas do código:

- MAX_TICKS(int): Guarda o valor máximo de ticks da simulação (25).
- INTERVALO_MS(int): Guarda o intervalo de 400 milissegundos entre cada tick

Estrutura do código

Estados A

- Classe EstadoAtacando: Implementa o estado de ataque, executa o estado, imprime a entrada e saída e transiciona para “EstadoRepouso”.
- Classe EstadoBuscando: Implementa o estado de busca, executa o estado, imprime a entrada e saída e transiciona para “EstadoAtacando”.
- Classe EstadoRepouso: Implementa o estado de ataque, executa o estado, imprime a entrada e saída e transiciona para “EstadoBuscando”.

Estados B

- Classe EstadoAlerta: Implementa o estado de ataque, executa o estado, imprime a entrada e saída e transiciona para “EstadoPatrulhando”.
- Classe EstadoPatrulhando: Implementa o estado de ataque, executa o estado, imprime a entrada e saída e transiciona para “EstadoAlerta”.

Outros

- Classe Agente: Recebe o nome do agente, define o estado inicial do agente, chama a entrada do agente, troca o estado atual do agente para outro, aumenta o contador de ticks e executa o comportamento do estado.
- Interface Estado: Define os três comportamentos obrigatórios dos estados: entrada(), executar() e saída().
- Classe Gerenciador: Cria os agentes, define seus estados iniciais, faz o loop de ticks, faz os agentes executarem os estados em cada tick, faz um intervalo entre os ticks.

Resultados esperados - Ex logs

[Agente A] inicializado no estado EstadoRepouso

>> [A] ENTRADA: Repouso (aguardando 3 ticks)

[Agente B] inicializado no estado EstadoPatrulhando

>> [B] ENTRADA: Patrulhando (limite 5 ticks)

===== TICK 1 =====

[A] Repouso executando... tick 1/3

[B] Patrulhando área... tick 1/5

===== TICK 2 =====

[A] Repouso executando... tick 2/3

[B] Patrulhando área... tick 2/5

===== TICK 3 =====

[A] Repouso executando... tick 3/3

[Agente A] TRANSIÇÃO: EstadoRepouso -> EstadoBuscando

<< [A] SAÍDA: Repouso

>> [A] ENTRADA: Buscando (limite 4 ticks)

[B] Patrulhando área... tick 3/5

===== TICK 4 =====

[A] Buscando alvo... tick 1/4

[B] Patrulhando área... tick 4/5

===== TICK 5 =====

[A] Buscando alvo... tick 2/4

[B] Patrulhando área... tick 5/5

[Agente B] TRANSIÇÃO: EstadoPatrulhando -> EstadoAlerta

<< [B] SAÍDA: Patrulhando
>> [B] ENTRADA: Alerta (limite 3 ticks)

===== TICK 6 =====

[A] Buscando alvo... tick 3/4
[B] Em ALERTA! tick 1/3

[...Continua...]

===== TICK 25 =====

[A] Buscando alvo... tick 4/4
[Agente A] TRANSIÇÃO: EstadoBuscando -> EstadoAtacando
<< [A] SAÍDA: Buscando
>> [A] ENTRADA: Atacando (limite 2 ticks)
[B] Patrulhando área... tick 1/5

Simulação encerrada após 25 ticks.

Limitações

Os agentes do cosigo não interagem entre si, e os estados todos realizam ações bem parecidas. Com um pouco mais de tempo, nós poderíamos fazer o código ser um pouco mais complexo.